جزوه

Docker

Docker: بسته‌بندی و اجرای همان چهار سرویس

1. Dockerfile · A multi-stage Dockerfile for each runtime چندمرحله‌ای برای .NET، Node، Python و Java
2. Running several services with Docker Compose · اجرای چند سرویس با Docker Compose: شبکه‌ی داخلی، ترتیب، داده‌ی ماندگار
3. Configuration and secrets · پیکربندی و secret: بدون مقدار پیش‌فرض، شکست زودهنگام

درس 1 · docker

A multi-stage Dockerfile for each runtime

Dockerfile چندمرحله‌ای برای .NET، Node، Python و Java

dockerfile | multi-stage | layer cache | non-root

بعد از این درس می‌توانی

- برای هر چهار سرویس بک‌اند یک Dockerfile چندمرحله‌ای بنویسی: ساخت با SDK، اجرا با runtime کوچک.
- ترتیب COPY را طوری بچینی که نصب وابستگی‌ها cache بماند.
- کانتینر را با کاربر غیر root و یک HEALTHCHECK اجرا کنی.

سناریو از پروژه‌ی واقعی

ReBiomed در Control Panel یک api.Dockerfile دارد: مرحله‌ی build با npm ci و tsc، مرحله‌ی نهایی فقط dist و وابستگی‌های production، با USER node. ClinicReport هم همین را برای .NET دارد: sdk:10.0 برای publish و aspnet:10.0 برای اجرا، با USER app. در این درس چهار Dockerfile برای سرویس‌های examples می‌سازیم و هر چهار را واقعاً build و اجرا کرده‌ایم. حجم imageها: پایتون ۲۷۸ مگابایت، تایپ‌اسکریپت ۴۱۰، سی‌شارپ ۴۲۵ و جاوا ۵۸۷ مگابایت.

سؤال چرا دو مرحله؟ یک FROM کافی نیست؟

پاسخ SDK و کامپایلر و cache پکیج‌ها فقط برای ساختن لازم‌اند. در image نهایی هم حجم اضافه‌اند و هم سطح حمله. مرحله‌ی دوم فقط خروجی را کپی می‌کند.

سؤال چرا package.json یا csproj جدا از بقیه‌ی کد کپی می‌شود؟

پاسخ Docker هر دستور را یک لایه می‌کند و تا وقتی ورودی‌اش عوض نشده از cache استفاده می‌کند. وابستگی‌ها کم عوض می‌شوند و کد زیاد. با این ترتیب، تغییر یک خط کد نصب پکیج‌ها را دوباره اجرا نمی‌کند.

مفهوم‌ها

image پایه‌ی هر runtime

sdk.: NET برای build، aspnet برای اجرا. Node: یک image برای هر دو، با npm prune --omit=dev در مرحله‌ی اول ساخته و کپی می‌شود. Python: venv Java: maven با JDK برای build، eclipse-temurin:21-jre برای اجرا.

لایه‌ها و cache

اول فایل وابستگی‌ها (csproj، package-lock، requirements، pom) را کپی کن و نصب کن، بعد کد را. dockerignore. جلوی کپی شدن bin/obj، .venv، node_modules و target محلی را می‌گیرد؛ هم سرعت دارد، هم از کپی شدن باینری سیستم‌عامل اشتباه جلوگیری می‌کند.

کاربر غیر root

node image کاربر node دارد، .NET کاربر app با $APP_UID. در Python و Java کاربر را می‌سازیم. پوشه‌ی داده (data/) قبل از USER به همان کاربر داده می‌شود، وگرنه volume تازه قابل نوشتن نیست.

HEALTHCHECK

Docker هر چند ثانیه دستور را اجرا می‌کند و کانتینر را healthy یا unhealthy علامت می‌زند. compose و orchestratorها از آن برای ترتیب بالا آمدن و راه‌اندازی دوباره استفاده می‌کنند. image runtime دات‌نت curl ندارد، پس نصبش کرده‌ایم؛ در Node و Python از خود runtime استفاده می‌کنیم.

مقایسه

ساخت image	
C#	`mcr.microsoft.com/dotnet/sdk:10.0`
TypeScript	`node:22-slim`
Python	`python:3.12-slim`
Java	`maven:3.9-eclipse-temurin-21`

Node و Python image جدای SDK ندارند؛ ابزار ساخت داخل همان image است.

اجرا image	
C#	`mcr.microsoft.com/dotnet/aspnet:10.0`
TypeScript	`node:22-slim`
Python	`python:3.12-slim`
Java	`eclipse-temurin:21-jre`

Java و .NET image کوچک‌تری برای اجرا دارند؛ در Node و Python کوچکی از کپی نکردن ابزارها می‌آید.

نصب وابستگی cacheپذیر	
C#	`COPY *.csproj; dotnet restore`
TypeScript	`COPY package*.json; npm ci`
Python	`COPY requirements.txt; pip install`
Java	`COPY pom.xml; mvn dependency:go-offline`

npm ci به package-lock دقیق پایبند است؛ npm install نیست.

خروجی ساخت	
C#	`dotnet publish -o /out`
TypeScript	`npm run build && npm prune --omit=dev`
Python	`venv کامل /venv`
Java	`mvn package → target/*.jar`

Spring Boot یک jar اجرایی می‌سازد که همه‌ی وابستگی‌ها داخلش است.

کاربر غیر root	
C#	`USER $APP_UID`
TypeScript	`USER node`
Python	`useradd ... && USER app`
Java	`useradd ... && USER app`

imageهای .NET و Node کاربر آماده دارند.

health check	
C#	`curl -fsS .../status` (نصب‌شده)
TypeScript	`node -e "fetch(...)"`
Python	`python -c "urllib.request.urlopen(...)"`
Java	`curl -fsS .../status`

از ابزاری استفاده کن که از قبل در image هست، تا چیز اضافه نصب نکنی.

کد

Dockerfile

C# کد اجراشده و تست‌شده در /examples `examples/backend/csharp/Dockerfile`

```
# Stage 1: the SDK image restores and publishes.
FROM mcr.microsoft.com/dotnet/sdk:10.0 AS build
WORKDIR /src
# The project file first: restore stays cached until the packages change.
COPY Portfolio/Portfolio.csproj Portfolio/
RUN dotnet restore Portfolio/Portfolio.csproj
COPY Portfolio/ Portfolio/
RUN dotnet publish Portfolio/Portfolio.csproj -c Release -o /out --no-restore

# Stage 2: the much smaller ASP.NET runtime image, no SDK.
FROM mcr.microsoft.com/dotnet/aspnet:10.0
# The runtime image has no curl; the health check needs it.
RUN apt-get update && apt-get install -y --no-install-recommends curl && rm -rf /var/lib/apt/lists/* \
  && mkdir /data && chown $APP_UID /data
WORKDIR /app
COPY --from=build /out .
ENV ASPNETCORE_HTTP_PORTS=8080 DB_PATH=/data/portfolio.db
# APP_UID is the non-root "app" user the .NET images ship with.
USER $APP_UID
EXPOSE 8080
HEALTHCHECK --interval=10s --timeout=3s --retries=3 CMD curl -fsS http://127.0.0.1:8080/status || exit 1
ENTRYPOINT ["dotnet", "Portfolio.dll"]
```

TypeScript کد اجراشده و تست‌شده در /examples

examples/backend/typescript/Dockerfile

```
# Stage 1: install everything (TypeScript is a dev dependency) and compile.
FROM node:22-slim AS build
WORKDIR /app
# package files first: npm ci stays cached until the dependencies change.
COPY package*.json ./
RUN npm ci
COPY tsconfig.json ./
COPY src ./src
# Compile, then drop dev dependencies so only runtime packages get copied on.
RUN npm run build && npm prune --omit=dev

# Stage 2: compiled JavaScript and production node_modules only.
FROM node:22-slim
WORKDIR /app
ENV NODE_ENV=production PORT=8080
COPY --from=build /app/package.json ./
COPY --from=build /app/node_modules ./node_modules
COPY --from=build /app/dist ./dist
# The node image ships a non-root "node" user.
USER node
EXPOSE 8080
HEALTHCHECK --interval=10s --timeout=3s --retries=3 \
  CMD node -e "fetch('http://127.0.0.1:8080/status').then(r => process.exit(r.ok ? 0 : 1), () =>
process.exit(1))"
CMD ["node", "dist/main.js"]
```

Python کد اجراشده و تست‌شده در /examples

examples/backend/python/Dockerfile

```
# Stage 1: install dependencies into a virtualenv.
FROM python:3.12-slim AS build
RUN python -m venv /venv
# requirements.txt first: this layer stays cached until the dependencies change.
COPY requirements.txt .
RUN /venv/bin/pip install --no-cache-dir -r requirements.txt

# Stage 2: only the venv and the code, no pip cache or build tools.
FROM python:3.12-slim
ENV PATH=/venv/bin:$PATH PYTHONUNBUFFERED=1 DB_PATH=/data/portfolio.db
COPY --from=build /venv /venv
WORKDIR /app
COPY portfolio ./portfolio
# Never run as root; /data is the only place the app writes.
RUN useradd --uid 10001 app && mkdir /data && chown app /data
USER app
EXPOSE 8080
HEALTHCHECK --interval=10s --timeout=3s --retries=3 \
  CMD python -c "import urllib.request; urllib.request.urlopen('http://127.0.0.1:8080/status', timeout=2)"
CMD ["uvicorn", "portfolio.main:app", "--host", "0.0.0.0", "--port", "8080"]
```

Java کد اجراشده و تست‌شده در /examples `examples/backend/java/Dockerfile`

```
# Stage 1: Maven with a full JDK builds the jar.
FROM maven:3.9-eclipse-temurin-21 AS build
WORKDIR /app
# pom.xml first: the dependency download stays cached until the pom changes.
COPY pom.xml .
RUN mvn -B -q dependency:go-offline
COPY src ./src
RUN mvn -B -q -DskipTests package

# Stage 2: a JRE is enough to run it.
FROM eclipse-temurin:21-jre
RUN useradd --uid 10001 app && mkdir /data && chown app /data
WORKDIR /app
COPY --from=build /app/target/portfolio-1.0.0.jar app.jar
ENV DB_PATH=/data/portfolio.db SERVER_PORT=8080
USER app
EXPOSE 8080
HEALTHCHECK --interval=10s --timeout=3s --start-period=30s --retries=3 \
  CMD curl -fsS http://127.0.0.1:8080/status || exit 1
ENTRYPOINT ["java", "-jar", "app.jar"]
```

اشتباه‌های رایج

همه **COPY . . قبل از نصب وابستگی‌ها**

چرا: هر تغییر کد cache را می‌شکند و نصب پکیج‌ها هر بار از اول اجرا می‌شود.

درست: اول فایل وابستگی‌ها، بعد نصب، بعد کد.

همه **نداشتن .dockerignore**

چرا: node_modules یا .venv یا bin/obj ویندوزی وارد image لینوکسی می‌شود؛ build کند و گاهی خراب می‌شود.

درست: .dockerignore کنار هر Dockerfile.

همه **اجرا به‌عنوان root**

چرا: هر آسیب‌پذیری در اپ با دسترسی root داخل کانتینر اجرا می‌شود.

درست: USER غیر root؛ مالکیت پوشه‌های نوشتنی را قبلش بده.

TypeScript **کپی کل node_modules مرحله‌ی build بدون prune**

چرا: TypeScript، tsx و @types وارد image نهایی می‌شوند.

درست: npm prune --omit=dev یا npm ci --omit=dev در مرحله‌ی دوم (روش ReBiomed).

C# **HEALTHCHECK با curl روی aspnet بدون نصب آن**

چرا: image runtime curl ندارد؛ کانتینر همیشه unhealthy می‌شود.

درست: curl را نصب کن یا یک حالت --health در خود اپ بساز.

این کد کجای پروژه‌هاست

`ControlPanel/deploy/rebiomed/api.Dockerfile:4`

ReBiomed: build و runtime جدا، USER node، npm ci --omit=dev

`ClinicReport/Dockerfile:10`

sdk:10.0 برای ساخت، aspnet:10.0 برای اجرا، USER app

تمرین

▼ 1. یک خط به Program.cs اضافه کن و image را دوباره build کن. کدام مرحله‌ها از cache آمدند؟

پاسخ: COPY csproj و dotnet restore از cache آمدند (CACHED)؛ از COPY Portfolio/ به بعد دوباره اجرا شد.

▼ 2. در Dockerfile پایتون، تست‌ها و pytest هم در image هستند؟ چطور بیرونشان می‌گذاری؟

پاسخ: پوشه‌ی tests با .dockerignore بیرون است، ولی pytest در requirements.txt نصب می‌شود. requirements را به دو فایل (runtime و dev) تقسیم کن و فقط runtime را در Dockerfile نصب کن.

▼ 3. docker run --rm lessons-python id -u چه عددی برمی‌گرداند؟

پاسخ: 10001؛ کاربر app، نه (0) root.

جمله‌ی مصاحبه

Each service builds in a full SDK stage and runs from a small runtime image, with dependencies cached in their own layer, a non-root user and a health check.

درس 2 · docker

Running several services with Docker Compose

اجرای چند سرویس با Docker Compose: شبکه‌ی داخلی، ترتیب، داده‌ی ماندگار

docker compose | networks | healthchecks | volumes

بعد از این درس می‌توانی

- چهار سرویس و یک MongoDB را با یک فایل compose بالا بیاوری و منتظر healthy شدن همه بمانی.
- دیتابیس را فقط روی یک شبکه‌ی internal بگذاری تا از بیرون در دسترس نباشد.
- با depends_on و condition: service_healthy ترتیب درست را تضمین کنی و داده را در volume نگه داری.

سناریو از پروژه‌ی واقعی

compose.apps.yml در Control Panel سرویس rebiomed-mongo را روی شبکه‌ای می‌گذارد که فقط API ReBiomed به آن وصل است. Mongo هیچ پورتی روی میزبان باز نمی‌کند و API با depends_on: condition: service_healthy منتظر آن می‌ماند. compose درس ما همین الگو را برای چهار سرویس examples پیاده می‌کند. با docker compose up --build --wait همه build می‌شوند و دستور تا healthy شدن همه برمی‌گردد. ما این را اجرا کرده‌ایم و /status هر چهار سرویس روی پورت‌های 8081 تا 8084 جواب داد.

سؤال depends_on بدون condition کافی نیست؟

پاسخ depends_on ساده فقط صبر می‌کند تا کانتینر شروع شود، نه تا دیتابیس آماده‌ی پذیرش اتصال باشد. Mongo یا SQL Server چند ثانیه بعد از شروع آماده می‌شوند. condition: service_healthy منتظر healthcheck می‌ماند.

سؤال اگر پورت Mongo را باز نکنم، خودم چطور به آن وصل شوم؟

پاسخ docker compose exec mongo mongosh. برای توسعه‌ی محلی می‌شود یک فایل compose.override.yml با ports داشت که روی سرور استفاده نمی‌شود.

مفهوم‌ها

شبکه‌ها

سرویس‌های روی یک شبکه همدیگر را با نام سرویس پیدا می‌کنند (mongodb://mongo:27017). شبکه‌ی internal: true هیچ مسیری به بیرون ندارد و چیزی روی آن publish نمی‌شود. python روی هر دو شبکه است؛ بقیه فقط روی public.

ports و expose

ports: ["8081:8080"] پورت 8080 کانتینر را روی 8081 میزبان باز می‌کند. سرویسی که ports ندارد فقط از داخل شبکه‌های خودش در دسترس است. دیتابیس معمولاً نباید ports داشته باشد.

volume

پوشه‌ی /data هر سرویس و /data/db مونگو در volumeهای نام‌دار است و با down یا rebuild پاک نمی‌شود. فقط down -v آن‌ها را پاک می‌کند.

--wait

docker compose up --wait تا وقتی همه‌ی سرویس‌ها running و healthy شوند برمی‌گردد و اگر یکی unhealthy شود با خطا تمام می‌شود. برای CI و اسکریپت‌ها عالی است.

یک فایل، چهار زبان

compose به زبان سرویس اهمیت نمی‌دهد. هر سرویس build: پوشه‌ی خودش را دارد، با Dockerfile درس قبل. این دقیقاً همان الگوی adapterهاست: core-api فقط HTTP می‌بیند.

مقایسه

build سرویس	
C#	`build: ../backend/csharp`
TypeScript	`build: ../backend/typescript`
Python	`build: ../backend/python`
Java	`build: ../backend/java`

همه یکسان؛ تفاوت در Dockerfile هر پوشه است.

پورت میزبان	
C#	`8083:8080`
TypeScript	`8082:8080`
Python	`8081:8080`
Java	`8084:8080`

داخل کانتینر همه روی 8080 گوش می‌دهند؛ .NET با ASPNETCORE_HTTP_PORTS، Spring با SERVER_PORT، uvicorn با --port، Node با PORT.

داده‌ی ماندگار	
C#	`csharp-data:/data`
TypeScript	ندارد (در حافظه sql.js)
Python	`python-data:/data`
Java	`java-data:/data`

sql.js دیتابیس را در حافظه نگه می‌دارد؛ برای ماندگاری باید آن را export و در فایل ذخیره کرد.

زمان آماده شدن	
C#	چند ثانیه
TypeScript	کمتر از یک ثانیه
Python	حدود یک ثانیه
Java	۱۰ تا ۲۰ ثانیه (start-period: 30s)

Spring Boot و JVM کندتر بالا می‌آیند؛ start-period جلوی unhealthy شدن زودهنگام را می‌گیرد.

کد

compose.yml: چهار سرویس و یک MongoDB داخلی

همه‌ی زبان‌ها کد اجراشده و تست‌شده در /examples `examples/docker/compose.yml`

```yaml
services:
  python:
    build: ../backend/python
    environment: *secrets
    ports: ["8081:8080"]
    volumes: [python-data:/data]
    networks: [public, internal]
    depends_on:
      mongo:
        condition: service_healthy # wait until Mongo answers, not just until it started

  typescript:
    build: ../backend/typescript
    environment: *secrets
    ports: ["8082:8080"]
    networks: [public]

  csharp:
    build: ../backend/csharp
    environment: *secrets
    ports: ["8083:8080"]
    volumes: [csharp-data:/data]
    networks: [public]

  java:
    build: ../backend/java
    environment: *secrets
    ports: ["8084:8080"]
    volumes: [java-data:/data]
    networks: [public]

  mongo:
    image: mongo:7
    # No "ports": Mongo is not published on the host; only services on "internal" reach it.
    volumes: [mongo-data:/data/db]
    networks: [internal]
    healthcheck:
      test: ["CMD", "mongosh", "--quiet", "--eval", "db.adminCommand('ping').ok"]
      interval: 5s
      retries: 10

networks:
  public:
  internal:
    internal: true # no route to the outside world either

volumes:
  python-data:
  csharp-data:
  java-data:
  mongo-data:
```

همین الگو در Control Panel برای ReBiomed

همه‌ی زبان‌ها از کد واقعی پروژه (کوتاه‌شده)

```
# ControlPanel/compose.apps.yml (shortened): ReBiomed's own Mongo on a network only the API joins
  rebiomed-mongo:
    image: mongo:7
    volumes:
      - rebiomed-mongo:/data/db
    networks:
      - rebiomed
    healthcheck:
      test: ["CMD", "mongosh", "--quiet", "--eval", "db.adminCommand('ping').ok"]

  rebiomed-api:
    build:
      context: ${REBIOMED_DIR:?set REBIOMED_DIR in .env (the ReBiomed repo folder)}
      dockerfile: ${DEPLOY_DIR:?set DEPLOY_DIR in .env}/rebiomed/api.Dockerfile
    depends_on:
      rebiomed-mongo:
        condition: service_healthy
    environment:
      MONGO_URI: mongodb://rebiomed-mongo:27017/rebiomed
      JWT_SECRET: ${REBIOMED_JWT_SECRET:?set REBIOMED_JWT_SECRET in .env (32+ chars)}
```

اشتباه‌های رایج

همه **ports برای دیتابیس روی سرور**

چرا: دیتابیس از اینترنت قابل اسکن و حمله می‌شود.

درست: بدون ports، روی شبکه‌ی internal؛ برای توسعه یک override فایل.

همه **depends_on بدون condition**

چرا: اپ قبل از آماده شدن دیتابیس بالا می‌آید و اولین اتصال‌ها شکست می‌خورند.

درست: healthcheck روی دیتابیس و condition: service_healthy.

همه **localhost در connection string داخل کانتینر**

چرا: localhost داخل کانتینر خود کانتینر است، نه میزبان و نه سرویس دیگر.

درست: نام سرویس (db، mongo) یا host.docker.internal برای میزبان.

همه **docker compose down -v در عادت روزمره**

چرا: volumeها و همه‌ی داده‌ها پاک می‌شوند.

درست: down بدون -v؛ -v فقط وقتی عمداً می‌خواهی از صفر شروع کنی.

این کد کجای پروژه‌هاست

`ControlPanel/compose.apps.yml:56`

rebiomed-mongo روی شبکه‌ی خودش، بدون ports

`ControlPanel/compose.apps.yml:76`

condition: service_healthy

تمرین

▼ 1. از داخل کانتینر typescript به mongo وصل شو (مثلاً با node -e و net.connect). چه می‌شود و چرا؟

پاسخ: اتصال برقرار نمی‌شود (نام mongo حل نمی‌شود). typescript فقط روی شبکه‌ی public است و mongo فقط روی internal.

▼ 2. healthcheck مونگو را عمداً خراب کن (مثلاً دستور اشتباه). docker compose up --wait چه می‌کند؟

پاسخ: mongo هرگز healthy نمی‌شود، python شروع نمی‌شود و --wait بعد از timeout با خطا تمام می‌شود.

▼ 3. یک سرویس ششم اضافه کن که هر ۳۰ ثانیه /status هر چهار سرویس را با curl بخواند. کدام شبکه لازم دارد؟

پاسخ: public؛ و آدرس‌ها http://python:8080/status و ... با نام سرویس، نه localhost:8081.

جمله‌ی مصاحبه

One compose file runs all services: the database sits on an internal network with no published port, and apps wait for it to be healthy.

درس 3 · docker

Configuration and secrets

پیکربندی و secret: بدون مقدار پیش‌فرض، شکست زودهنگام

configuration | secrets | fail fast

بعد از این درس می‌توانی

- تنظیمات را از متغیر محیطی بخوانی و secretها را هرگز در کد یا image نگذاری.
- اپ را طوری بنویسی که بدون secret معتبر اصلاً بالا نیاید (fail fast).
- در compose با ${VAR:?message} همین قاعده را قبل از اجرای هر کانتینر اعمال کنی.

سناریو از پروژه‌ی واقعی

core-api در Control Panel قبل از هر کاری validateJwtSecrets را صدا می‌زند: اگر JWT_SECRET یا JWT_REFRESH_SECRET نباشد، کوتاه‌تر از ۳۲ کاراکتر باشد یا هر دو برابر باشند، اپ با پیامی که دستور ساخت secret را هم می‌گوید متوقف می‌شود. compose.apps.yml هم برای ReBiomed ${REBIOMED_JWT_SECRET:?...} دارد. ما هر دو لایه را ساخته و تست کرده‌ایم: بدون متغیر، docker compose config با همان پیام می‌ایستد.

سؤال یک مقدار پیش‌فرض برای توسعه بدی‌اش چیست؟

پاسخ مقدار پیش‌فرض در ریپو است و هر کسی آن را می‌بیند. روزی که کسی فراموش کند متغیر را روی سرور تنظیم کند، سرور با همان secret عمومی توکن امضا می‌کند و هیچ خطایی نمی‌بیند.

سؤال چرا چک در اپ وقتی compose خودش چک می‌کند؟

پاسخ چون اپ همیشه با compose اجرا نمی‌شود: dotnet run، تست، Kubernetes، یک سرور دیگر. compose فقط وجود را چک می‌کند؛ اپ طول و متفاوت بودن را هم.

مفهوم‌ها

تنظیم از محیط

همه‌ی چیزهایی که بین محیط‌ها فرق می‌کند (connection string، secret، پورت) از متغیر محیطی می‌آیند (اصل سوم Twelve-Factor). image یکی است و در dev و prod فقط env فرق می‌کند.

Fail fast

چک در لحظه‌ی شروع، نه در اولین login. خطا در deploy دیده می‌شود، نه ساعت‌ها بعد توسط یک کاربر. پیام خطا بگوید چه کاری باید کرد.

تولید secret

node -e یا "python -c "import secrets; print(secrets.token_urlsafe(48))

"console.log(require('crypto').randomBytes(48).toString('base64'))"". فایل .env در .gitignore باشد و فقط روی سرور ساخته شود (make-env.sh در deploy/vps).

${VAR:?msg} در compose

اگر VAR خالی یا تعریف‌نشده باشد، compose با پیام msg متوقف می‌شود. ${VAR:-default} برعکس آن است: مقدار پیش‌فرض می‌دهد. برای secret هرگز :-.

مقایسه

خواندن متغیر محیطی	
C#	`Environment.GetEnvironmentVariable("X")`
TypeScript	`process.env.X`
Python	`os.environ.get("X")`
Java	`System.getenv("X")`

ASP.NET و Spring همچنین متغیرها را به سیستم پیکربندی خود نگاشت می‌کنند (Section__Key، SECTION_KEY).

اعتبارسنجی در شروع	
C#	`Build قبل از Config.Load(env)`
TypeScript	`listen قبل از loadSettings()`
Python	`import main هنگام load_settings()`
Java	`@Configuration در Config.load(System.getenv())`

در Spring خطای سازنده‌ی bean کل context را متوقف می‌کند، که همان fail fast است.

معادل فریم‌ورک	
C#	`IOptions<T> + ValidateOnStart()`
TypeScript	`@nestjs/config + validation schema`
Python	`pydantic-settings BaseSettings`
Java	`@ConfigurationProperties + @Validated`

در پروژه‌های بزرگ‌تر از این‌ها استفاده کن؛ ایده همان است.

ورود secret در compose	
C#	`${JWT_SECRET:?message}`
TypeScript	`${JWT_SECRET:?message}`
Python	`${JWT_SECRET:?message}`
Java	`${JWT_SECRET:?message}`

یکسان؛ compose به زبان اهمیتی نمی‌دهد.

کد

خواندن و چک تنظیمات

C# کد اجراشده و تست‌شده در /examples `examples/backend/csharp/Portfolio/Config.cs`

```
public static string RequireSecret(IDictionary<string, string?> env, string name)
{
    // No fallback value: a missing secret stops the app at startup, not on the first login.
    var value = env.TryGetValue(name, out var v) ? v ?? "" : "";
    if (value.Length < MinSecretLength)
        throw new InvalidOperationException($"{name} must be set to a random value of at least
{MinSecretLength} characters.");
    return value;
}

public static Settings Load(IDictionary<string, string?> env)
{
    var access = RequireSecret(env, "JWT_SECRET");
    var refresh = RequireSecret(env, "JWT_REFRESH_SECRET");
    if (access == refresh)
        throw new InvalidOperationException("JWT_SECRET and JWT_REFRESH_SECRET must be different values.");
    return new(access, refresh, env.TryGetValue("DB_PATH", out var db) && db is not null ? db :
"portfolio.db");
}
```

examples/backend/typescript/src/config.ts کد اجراشده و تست‌شده در /examples **TypeScript**

```ts
export function requireSecret(env: Record<string, string | undefined>, name: string): string {
  // No fallback value: a missing secret stops the app at startup, not on the first login.
  const value = env[name] ?? '';
  if (value.length < MIN_SECRET_LENGTH)
    throw new Error(`${name} must be set to a random value of at least ${MIN_SECRET_LENGTH} characters.`);
  return value;
}

export function loadSettings(env: Record<string, string | undefined> = process.env) {
  const jwtSecret = requireSecret(env, 'JWT_SECRET');
  const jwtRefreshSecret = requireSecret(env, 'JWT_REFRESH_SECRET');
  if (jwtSecret === jwtRefreshSecret) throw new Error('JWT_SECRET and JWT_REFRESH_SECRET must be different
values.');
  return { jwtSecret, jwtRefreshSecret, port: Number(env.PORT ?? 8080) };
}
```

examples/backend/python/portfolio/config.py کد اجراشده و تست‌شده در /examples **Python**

```python
def require_secret(env: Mapping[str, str], name: str) -> str:
    # No fallback value: a missing secret stops the app at startup, not on the first login.
    value = env.get(name, "")
    if len(value) < MIN_SECRET_LENGTH:
        raise RuntimeError(f"{name} must be set to a random value of at least {MIN_SECRET_LENGTH} characters.")
    return value


def load_settings(env: Mapping[str, str] = os.environ) -> dict[str, str]:
    access = require_secret(env, "JWT_SECRET")
    refresh = require_secret(env, "JWT_REFRESH_SECRET")
    if access == refresh:
        raise RuntimeError("JWT_SECRET and JWT_REFRESH_SECRET must be different values.")
    return {"jwt_secret": access, "jwt_refresh_secret": refresh, "db_path": env.get("DB_PATH", "portfolio.db")}
```

examples/backend/java/src/main/java/portfolio/Config.java کد اجراشده و تست‌شده در /examples **Java**

```java
public static String requireSecret(Map<String, String> env, String name) {
    // No fallback value: a missing secret stops the app at startup, not on the first login.
    var value = env.getOrDefault(name, "");
    if (value.length() < MIN_SECRET_LENGTH)
        throw new IllegalStateException(name + " must be set to a random value of at least " +
MIN_SECRET_LENGTH + " characters.");
    return value;
}

public static Settings load(Map<String, String> env) {
    var access = requireSecret(env, "JWT_SECRET");
    var refresh = requireSecret(env, "JWT_REFRESH_SECRET");
    if (access.equals(refresh))
        throw new IllegalStateException("JWT_SECRET and JWT_REFRESH_SECRET must be different values.");
    return new Settings(access, refresh, env.getOrDefault("DB_PATH", "portfolio.db"));
}
```

secret اجباری در compose

همه‌ی زبان‌ها کد اجراشده و تست‌شده در /examples examples/docker/compose.yml

```
# One block of settings shared by every service (a YAML anchor). ${VAR:?message} stops
# `docker compose up` with that message when the variable is missing: no default secrets.
x-secrets: &secrets
  JWT_SECRET: ${JWT_SECRET:?set JWT_SECRET to a random value of 32+ characters}
  JWT_REFRESH_SECRET: ${JWT_REFRESH_SECRET:?set JWT_REFRESH_SECRET to a different random value of 32+
characters}
```

اشتباه‌های رایج

همه **ENV JWT_SECRET=... در Dockerfile**

چرا: secret داخل لایه‌ی image ذخیره می‌شود و با docker history یا هر کسی که image را pull کند دیده می‌شود.

درست: secret فقط هنگام اجرا: environment در compose از .env، یا Docker/Kubernetes secrets.

همه **commit کردن فایل .env**

چرا: secretها در تاریخچه‌ی git می‌مانند، حتی بعد از پاک کردن فایل.

درست: .env در .gitignore؛ یک .env.example بدون مقدار در ریپو.

همه **لاگ کردن تنظیمات در شروع**

چرا: secretها در لاگ و سیستم‌های جمع‌آوری لاگ پخش می‌شوند.

درست: فقط نام تنظیم یا «set / not set» را لاگ کن.

همه **${JWT_SECRET:-dev-secret} در compose**

چرا: روی سرور هم بی‌صدا از secret عمومی استفاده می‌شود.

درست: ${JWT_SECRET:?...}.

این کد کجای پروژه‌هاست

ControlPanel/services/core-api/src/auth/jwt-config.ts:13

requireSecret: بدون مقدار پیش‌فرض، حداقل ۳۲ کاراکتر

ControlPanel/services/core-api/src/main.ts:11

validateJwtSecrets قبل از listen

ControlPanel/compose.apps.yml:79

${REBIOMED_JWT_SECRET:?...}

تمرین

▼ 1. env -u JWT_SECRET docker compose -f examples/docker/compose.yml config را اجرا کن. چه می‌بینی؟

پاسخ: error while interpolating ... required variable JWT_SECRET is missing a value: set JWT_SECRET to a random value of 32+ characters. هیچ کانتینری ساخته نمی‌شود.

▼ 2. JWT_SECRET و JWT_REFRESH_SECRET را برابر بگذار و سرویس java را بالا بیاور. کجا متوقف می‌شود؟

پاسخ: compose آن را قبول می‌کند (فقط وجود را چک می‌کند)، ولی Spring در ساخت bean کلاس Real با IllegalStateException متوقف می‌شود و کانتینر خارج می‌شود.

▼ 3. یک فایل .env.example برای این compose بنویس.

پاسخ: JWT_SECRET= JWT_REFRESH_SECRET= # generate each with: python -c "import secrets; print(secrets.token_urlsafe(48))"

جمله‌ی مصاحبه

Configuration comes from the environment, secrets have no defaults, and both compose and the app refuse to start without valid ones.